FROST Guide

Threshold RedPallas: FROST, its re-randomised form, and where Zcash uses it

m@rek.onl

Abstract

Assuming the RedPallas signatures of the *Ironwood Guide*, this volume of *The Zcash Arboretum* documents threshold signing for Zcash: FROST as its paper constructs it and as RFC 9591 standardises it, the re-randomised form that survives RedPallas key re-randomisation, and the surfaces that use it — spend authorisation, recoverability custody, and finalizer keys. The audit boundary is stated plainly: the deployed re-randomised path has not been audited, and every claim beyond the paper, the RFC, and the draft ZIP carries its classification.

Contents

1 Scope, sources, and deployment surfaces

This volume documents threshold custody of Zcash spend authority: how t of n parties, no one of whom ever holds the whole signing key, jointly produce a signature that consensus accepts as an ordinary RedPallas spend-authorisation signature. Three artefacts carry the construction. The base scheme is FROST, the two-round threshold Schnorr protocol of Komlo and Goldberg, standardised as RFC 9591 (§2). The Zcash layer is Re-Randomized FROST (Gouvêa–Komlo, IACR ePrint 2024/436), which shifts every share by a common randomizer so that the aggregate signature verifies under the per-Action randomised key; ZIP 312 specifies it for Zcash, and the `frost-rerandomized` and `reddsa` crates ship it (§3). The deployment picture — the running stack, the proposed uses, and the gaps — closes the volume (§4).

Three custody surfaces anchor the treatment, one per rigor bin (§1.3): spend-authorisation custody for Orchard, where the construction is specified and shipped; custody of the ZIP-2005 quantum spending key qsk, where a Proposed consensus ZIP constrains a FROST ceremony whose recovery protocol it defers; and Crosslink finalizer keys, where no threshold design exists at all. The surfaces and their bins are fixed in §1.4.

1.1 Scope

The object under study is a t-of-n signing protocol whose output a verifier cannot tell apart from single-party spend authorisation. ZIP 312 states the requirements directly: the signatures must verify as Sapling or Orchard spend-authorisation signatures, and should meet the protocol specification's "Signature with Re-Randomizable Keys" criteria. Plain FROST fails the first clause — it signs for a fixed group key, and consensus verifies only per-Action randomised keys — and closing that gap is the volume's central construction (§3).

The volume develops RedPallas only, per its title. ZIP 312 itself defines two ciphersuites — FROST(Jubjub, BLAKE2b-512) for Sapling RedJubjub and FROST(Pallas, BLAKE2b-512) for Orchard RedPallas — and we cite the Jubjub suite where the ZIP fixes both at once, but develop only the Pallas suite (§3.5).

ZIP 312's stated non-requirements bound the scope from below: it does not remove the Coordinator role; it defines no key-generation procedure (out of scope there, exactly as in RFC 9591); and it provides no network privacy. Key generation nevertheless appears twice in this volume, both times forced by sources: once as published with the base scheme (§2.2), and once as the unspecified-but-shipped DKG surface of the `reddsa` crate, binned in §1.3.

One altitude note governs every deployment claim. The deployment is library-level, never consensus-level: no consensus rule knows FROST exists, and none needed to change. The verifier accepts the aggregate because it is a valid RedPallas signature; as ZIP 312's rationale puts it, the method by which the commitment R was generated is irrelevant to the verifier.

1.2 The source ladder

Table 1 pins every primary source. All repository facts in this volume were verified firsthand at the stated commits; web sources were fetched 2026-07-15; local copies of the papers and the RFC are archived with the volume's drafts.

The standard. RFC 9591, "The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures" (D. Connolly, Zcash Foundation; C. Komlo, University of Waterloo and Zcash Foundation; I. Goldberg, University of Waterloo; C. A. Wood, Cloudflare; IRTF CFRG stream, Category Informational, June 2024), is the top of the ladder and the only artefact here at standards rigor. Two of its four authors are Zcash Foundation: the standard and the Zcash deployment share provenance. It fixes the base two-round protocol and leaves key generation out of scope; its deltas against the SAC 2020 paper are catalogued in §2.8.

Artefact	Pin / date	Rigor
RFC 9591 (IRTF CFRG)	June 2024	Informational; standards rigor
ePrint 2024/436 (Gouvêa–Komlo)	recv. 2024-03-13	preprint; no venue
ZIP 312	`6961098410`, 2026-07-09	Draft (Wallet category)
ZIP 2005	`6961098410`, 2026-07-09	Proposed (Consensus category)
`ZcashFoundation/frost`	`2016e44ba4`, 2026-04-23	shipped code; workspace 3.0.0
`ZcashFoundation/reddsa`	`3792daa95e`, 2026-05-22	shipped code; crate 0.5.2
ePrint 2020/852; 2022/833	SAC 2020; CRYPTO 2022	peer-reviewed antecedents

Table 1: Primary FROST sources, pinned. Verification date 2026-07-15.

The paper. "Re-Randomized FROST", by Gouvêa and Komlo (both Zcash Foundation), is IACR ePrint 2024/436 (`https://eprint.iacr.org/2024/436`): received 2024-03-13, approved 2024-03-15, publication info "Preprint", no peer-reviewed venue as of 2026-07-15. We read it in full and work from it throughout §3. Its headline result, Theorem 1, proves unforgeability in the random-oracle model under AOMDL — the same assumptions as plain FROST (§3.3). The construction it describes is specified and shipped; the security theorem above it rests on a preprint, and we say so wherever we lean on it.

The ZIP. ZIP 312, "FROST for Spend Authorization Multisignatures" (owners Conrado Gouvêa, Chelsea Komlo, and Deirdre Connolly; Status Draft, Category Wallet; Discussions-To zcash/zips#382, pull request zcash/zips#662), is the Zcash specification of the construction.[1] It is visibly unfinished — its Created field still reads `2022-08-dd`, day unfilled — so we cite it as a moving object, by repository commit (`zcash/zips` @ `6961098410`, 2026-07-09) and Draft status, never as a stable specification. Its threat model trusts the Coordinator and every key-share holder with transaction privacy and unlinkability, never with security: a rogue Coordinator cannot create a signed transaction without the approval of `MIN_PARTICIPANTS` participants. What it specifies — the unlinkability requirement, the randomizer flow, and the two ciphersuites — is developed in §3.4 and §3.5.

The crates. Repository `ZcashFoundation/frost` (@ `2016e44ba4`, 2026-04-23) is a Rust workspace at version 3.0.0 (`rust-version` 1.81): crate `frost-core` carries the ciphersuite-generic protocol, six ciphersuite crates instantiate it (ed25519, ed448, P-256, ristretto255, secp256k1, secp256k1-tr), `frost-rerandomized` carries the re-randomised layer, and `gencode` generates the per-suite code. The repository also vendors an audit report dated 2021-03-23 (`zcash-frost-audit-report-20210323.pdf`) and the source of the ZF FROST book (published at `frost.zfnd.org`), whose "FROST with Zcash" chapters (zcash-devtool demo, Ywallet demo, FROST server) and serialisation-format page — the page ZIP 312 cites for `encode_signing_package` — feed §4. Repository `ZcashFoundation/reddsa` (@ `3792daa95e`, 2026-05-22; crate version 0.5.2) holds the Zcash ciphersuites: its `frost` feature enables `frost-rerandomized` 3.0.0, and `src/frost/redpallas.rs` defines FROST(Pallas, BLAKE2b-512) as struct `PallasBlake2b512`, ZIP 312's reference implementation surface (§3.6). Crate `reddsa` also ships machinery ZIP 312 does not specify — DKG hash personalisations and even-Y normalisation hooks — binned in §1.3.

The antecedent papers. FROST itself is Komlo–Goldberg, IACR ePrint 2020/852 (SAC 2020); the security analysis the re-randomised extraction argument extends is Bellare–Crites–Komlo–Maller–Tessaro–Zhu, "Better than Advertised Security for Non-Interactive Threshold Signatures" (CRYPTO 2022; ePrint 2022/833). They are the paper's references [10] and [1] respectively; §2 works from both.

[1] The FROST ZIP is ZIP 312 alone. ZIP 313, its numeric neighbor, is "Reduce Conventional Transaction Fee to 1000 zatoshis" — Status Obsolete, obsoleted by ZIP 317 — and has nothing to do with FROST.

1.3 The three-bin rule

The series rule is binary for deployed behaviour and ternary for design. A claim about deployed behaviour cites the implementation — crate and file — and the specification section it realises; a claim about a design-stage protocol is classified *specified*, *designed but unspecified*, or an *open problem*, and the bin stays visible in the prose. This volume sits astride the boundary: one surface is shipped code, one is a consensus proposal, one is an acknowledged gap. Applied once, here, for the whole volume:

1. Re-Randomized FROST for spend-authorisation custody is *specified*: RFC 9591 published June 2024, ZIP 312 in Draft, ePrint 2024/436, and shipped code — `frost-rerandomized` 3.0.0 and `reddsa` 0.5.2. The deployment is library-level, not consensus (§1.1).

2. Custody of the ZIP-2005 quantum spending key is specified at the ZIP level (Status Proposed, Category Consensus), but the Recovery Protocol it exists to serve is "to be introduced, potentially, in a future upgrade" — *designed but unspecified*.

3. Crosslink finalizer threshold custody is an *open problem*: the *Crosslink Guide*'s own classification, which we adopt.

One surface refuses the clean split, and we bin it here once. No ZIP specifies FROST key generation: ZIP 312 declares it out of scope (as does RFC 9591), and ZIP 271's May-2025 note confirms the gap — ZIP 312's missing key-generation text is its stated reason not to couple disbursement of the *lockbox* (the in-protocol reserve that has accrued a portion of each block subsidy since NU6, per ZIP 2001) to that specification. Yet crate `reddsa` ships DKG support: hash personalisations `FROST_RedPallasD` (HDKG) and `FROST_RedPallasI` (HID), beyond ZIP 312's H1–H5 and HR, plus even-Y normalisation hooks (`post_generate`/`post_dkg`, applying `into_even_y` to secret shares and the public key package) so that the group verifying key meets the encoding constraint of Orchard's spend validating key. Deployed code with no specification above it: we classify the DKG surface *designed but unspecified* and cite the crate at commit `3792daa95e` wherever we rely on it.

1.4 The three deployment surfaces

Spend-authorisation custody (specified). The product ZIP 312 promises is signatures "indistinguishable from RedPallas Orchard Spend Authorization Signatures". The single-party object being imitated is settled in the *Ironwood Guide*: §"Spend authorisation: RedPallas re-randomisation" constructs $\mathsf{rsk} := \mathsf{ask} + \alpha$ and $\mathsf{rk} := \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\, G^{\mathsf{SpendAuth}}$, with GenRandom hashing 80 uniform bytes through $H^{\circledR}$, and §"RedPallas unforgeability" proves EUF-CMA from discrete log in the random-oracle model. Per the layering rule we cite both and re-prove neither; this volume's work begins where the single signer ends, at splitting ask into shares no signing coalition ever reassembles (§3). The wallet-side integration path is likewise on the record: the *Wallet Guide*, §"Partially created transactions (PCZT)", records ZIP 374's motivation (iii), threshold signing (for example FROST, RFC 9591) — the multiparty transaction format FROST signing rides in §4.3.

Custody of the quantum spending key (designed but unspecified). ZIP 2005, "Ironwood Quantum Recoverability" (owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; created 2025-03-31), writes FROST into its requirements twice: the design should be fully compatible with FROST threshold multisignatures (citing ZIP 312), and should lose no pre-quantum security when FROST and/or hardware wallets are used. Its "Usage with FROST" section then constrains the ceremony: the key ak is derived jointly, via the FROST DKG, from shares of ask; the participants must privately agree on a value sk and use it with $\mathsf{use\_qsk} = \mathsf{true}$ to derive nk, qsk, qk, and $\mathsf{rivk\_ext} = \mathsf{H}^{\mathsf{rivk\_ext}}_{\mathsf{qk}}(\mathsf{ak}, \mathsf{nk})$; all participants must obtain the same ak, nk, and rivk_ext; and each participant must hold qsk at least as securely as ask. The Recovery Protocol requires a RedDSA signature under ak — preserving t-of-n for as long as discrete log on Pallas stands — together with a proof $\mathrm{SoK}^{\mathsf{qsk}}$ of knowledge of a qsk with $\mathsf{H}^{\mathsf{qk}}(\mathsf{qsk}) = \mathsf{qk}$. The asymmetry is the point to retain: the key

qsk is held by *every* participant, so a quantum adversary needs only qsk and the threshold property is gone; the ZIP accordingly recommends moving funds to a fully post-quantum threshold protocol once one is available. The deferred Recovery Protocol is what parks this surface in the middle bin; the full treatment is §4.7.

Crosslink finalizer keys (open problem). The *Crosslink Guide*, §"Finalizer keys", records the prototype state: one ed25519 key per finalizer (`ed25519-zebra`), a 76-byte vote format signed by appending a 64-byte ed25519 signature over those 76 bytes, and — by negative search over the four Crosslink repositories (tfl-book, crosslink-protocol-guide, crosslink-spec, zebra-crosslink), dated 2026-07-15 there — no FROST or threshold-signing design for finalizer keys sourced anywhere in the corpus. That guide's classification bins threshold custody, key rotation, revocation, and initial-roster formation as open problems, and observes that FROST's presence in the PCZT flow makes the absence "a visible gap rather than an exotic ask". We adopt the classification and return to the surface in §4.8.

Adjacent surfaces, cited but not developed. Three further ZIPs touch FROST and mark how wide the surface is growing. ZIP 271 ("Dev Fund Extension and One-Time Disbursement", Proposed) anticipates spending lockbox one-time-disbursement chunks "to a FROST address", and its May-2025 note that ZIP 312 does not specify key generation is its stated reason not to couple the two specifications. ZIP 326 ("NU6.3 Consequences for Wallets", Draft, created 2026-06-30) cites ZIP 2005's "Usage with FROST" section. ZIP 374 ("Partially Created Zcash Transaction Format", Draft, created 2024-12-09) lists FROST per RFC 9591 under its multiparty-transactions motivation. Each appears in this volume exactly where it bears (§4); none is developed as a surface of its own.

2 FROST

The base object of this volume is the threshold Schnorr scheme FROST (*Flexible Round-Optimized Schnorr Threshold* signatures) of Komlo and Goldberg, published at SAC 2020 and maintained as IACR ePrint 2020/852; we work from the final revision of 22 December 2020. The scheme lets any t of n parties, each holding a Shamir share of a signing key s, produce a signature (R, z) that verifies under the ordinary single-party Schnorr equation for the group public key $Y = g^s$ — a verifier cannot distinguish a FROST signature from a single-signer one. That indistinguishability is precisely what makes the scheme deployable under Zcash's RedDSA validators: the consensus rules never learn that a threshold ceremony occurred. The deployment surfaces this volume targets are spend-authorisation custody over RedPallas (the *Ironwood Guide*, §"Spend authorisation: RedPallas re-randomisation"), custody of the ZIP-2005 quantum spending key (whose "Usage with FROST" section constrains the key generation), and Crosslink finalizer keys (the *Crosslink Guide*, §"Finalizer keys").

Throughout, $\mathbb{G}$ is a group of prime order q with generator g, and H, H_1, H_2 are hash functions with outputs in $\mathbb{Z}_q^*$, modelled as random oracles. The paper fixes the single-party scheme it must match — the Schnorr signature scheme of the *Crypto Guide*, §"From identification to signature via the Fiat–Shamir transform" — restated here in the paper's multiplicative notation: a Schnorr signature over m under secret key $s \in \mathbb{Z}_q$ and public key $Y = g^s \in \mathbb{G}$ is produced by sampling a nonce $k \xleftarrow{\$} \mathbb{Z}_q$, setting $R = g^k$, $c = H(R, Y, m)$, $z = k + s \cdot c$, and outputting $\sigma = (R, z)$; verification recomputes c and accepts iff $R = g^z \cdot Y^{-c}$. FROST's whole design problem is to compute the pair (R, z) jointly, with k and s existing only as shares, without opening the concurrency attacks described in §2.4.

2.1 Shamir sharing and share conversion

Two secret-sharing mechanisms cooperate in FROST, and keeping them distinct is the key to reading the signing equation.

Shamir sharing (the long-lived key). A dealer — or, in §2.2, every participant acting as a dealer simultaneously — selects $t-1$ random coefficients $a_1, \ldots, a_{t-1}$ and forms the degree-$(t-1)$ polynomial

$$f(x) = s + \sum_{i=1}^{t-1} a_i x^i, \qquad f(0) = s,$$

giving participant P_i the share $(i, f(i))$. Any t shares determine f by Lagrange interpolation and hence $s = f(0)$; fewer than t reveal nothing. For a signing set $S = \{p_1, \ldots, p_\alpha\}$ of α participant identifiers ($t \le \alpha \le n$), the interpolation weight of participant i at zero is the Lagrange coefficient

$$\lambda_i = \prod_{j=1, j\neq i}^{\alpha} \frac{p_j}{p_j - p_i},$$

computable from the identifiers alone — the coalition can be chosen *after* shares are dealt, which is what makes FROST's "any t of n, decided at signing time" flexibility possible.

Additive sharing and conversion (the per-signature nonce). A sum $s = \sum_i s_i$ with each summand chosen locally is an additive sharing: it needs no dealer and no interaction. The Cramer–Damgård–Ishai share conversion observation, used by FROST in its simplest case, is that additive shares convert locally to Shamir form: if $s = \sum_{i\in S} s_i$, then the values s_i/λ_i are Shamir shares of the same s (evaluations of a degree-$(\alpha-1)$ polynomial interpolating them). FROST uses exactly this to make the group nonce k: each signer contributes an additively-shared summand non-interactively, and the λ_i weights in the signing equation perform the conversion so that the response aggregates by plain summation. No DKG runs at signing time.

2.2 Key generation: Pedersen's DKG with a knowledge proof

FROST generates its long-lived shares with a variant of Pedersen's DKG — n parallel executions of Feldman's verifiable secret sharing, one with each participant as dealer — hardened with a proof of knowledge of each dealer's free coefficient. The paper's Figure 1, reproduced:

Round 1.

1. Every participant P_i samples t random values $(a_{i0}, \ldots, a_{i(t-1)}) \xleftarrow{\$} \mathbb{Z}_q$ and defines the degree-$(t-1)$ polynomial $f_i(x) = \sum_{j=0}^{t-1} a_{ij} x^j$.

2. Every P_i computes a proof of knowledge of a_{i0}: with $k \xleftarrow{\$} \mathbb{Z}_q$, $R_i = g^k$, $c_i = H(i, \Phi, g^{a_{i0}}, R_i)$, $\mu_i = k + a_{i0} \cdot c_i$, set $\sigma_i = (R_i, \mu_i)$, where Φ is a context string preventing replay across ceremonies.

3. Every P_i computes the public commitment $\vec{C_i} = \langle \phi_{i0}, \ldots, \phi_{i(t-1)} \rangle$ with $\phi_{ij} = g^{a_{ij}}$.

4. Every P_i broadcasts $\vec{C_i}, \sigma_i$.

5. On receiving $\vec{C_\ell}, \sigma_\ell$, participant P_i verifies $R_\ell \overset{?}{=} g^{\mu_\ell} \cdot \phi_{\ell 0}^{-c_\ell}$ with $c_\ell = H(\ell, \Phi, \phi_{\ell 0}, R_\ell)$, *aborting on failure*; on success all σ_ℓ are deleted.

Round 2.

1. Each P_i securely sends each other participant P_ℓ the share $(\ell, f_i(\ell))$, then deletes f_i and every share except its own $(i, f_i(i))$.

2. Each P_i verifies each received share against the dealer's commitment:

$$g^{f_\ell(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} \phi_{\ell k}^{i^k \bmod q},$$

aborting if the check fails.

3. Each P_i computes its long-lived signing share $s_i = \sum_{\ell=1}^{n} f_\ell(i)$, stores it securely, and deletes the $f_\ell(i)$.

4. Each P_i computes its public verification share $Y_i = g^{s_i}$ and the group public key $Y = \prod_{j=1}^{n} \phi_{j0}$.

Three design decisions deserve comment. First, the *proof of knowledge of* a_{i0} is the delta over plain Pedersen DKG: it forecloses rogue-key attacks in the $t \geq n/2$ regime, where a participant could otherwise choose its commitment as a function of others' to bias the aggregate key (the fix was suggested to the authors by Shlomovits and Turkel; the paper's changelog dates it to the 2020-07-08 revision). Second, the *complaint handling* is abort-only: where Gennaro et al. add a complaint/disqualification phase to make the DKG robust, FROST aborts on any failed share check and expects the cause to be investigated out of band; the paper notes that implementations wanting robustness can substitute the Gennaro et al. construction. Third, a *consistent view* of the broadcast commitments $\vec{C}_i$ across participants is assumed, not provided — the paper defers to whatever broadcast mechanism the deployment supplies. Each participant ends holding (i, s_i), with $Y_i = g^{s_i}$ public for share verification during signing and Y the single key the outside world sees.

2.3 Preprocessing and the two-round signing protocol

Signing splits into a message-independent *preprocess* stage and a single online round (equivalently, run as one two-round protocol with the commitment exchange inlined). A semi-trusted *signature aggregator* SA coordinates: it is trusted only for liveness and correct misbehaviour reporting, not for unforgeability — a malicious SA can at worst deny service or falsely accuse, never learn the private key or cause improper messages to be signed.

Preprocess(π) (paper Figure 2). Each participant P_i generates π single-use nonce pairs and publishes their commitments: for $1 \leq j \leq \pi$,

$$(d_{ij}, e_{ij}) \stackrel{\$}{\leftarrow} \mathbb{Z}_q^* \times \mathbb{Z}_q^*, \qquad (D_{ij}, E_{ij}) = (g^{d_{ij}}, g^{e_{ij}}),$$

appending (D_{ij}, E_{ij}) to a list L_i published to a predetermined location, and storing the secret halves for later.

Sign(m) (paper Figure 3). The aggregator SA selects a signing set S of α participants ($t \leq \alpha \leq n$), fetches each member's next unused commitment pair, and forms the ordered list $B = \langle (i, D_i, E_i) \rangle_{i \in S}$, sending (m, B) to every $P_i, i \in S$. Each participant then:

1. validates m (a signer signs only messages it is willing to sign) and checks $D_\ell, E_\ell \in \mathbb{G}^*$ for every commitment in B, aborting on failure;

2. computes the *binding values*

$$\rho_\ell = H_1(\ell, m, B), \quad \ell \in S,$$

the group commitment and challenge

$$R = \prod_{\ell \in S} D_\ell \cdot (E_\ell)^{\rho_\ell}, \qquad c = H_2(R, Y, m);$$

3. computes its response, converting its Shamir share to the additive aggregation domain via λ_i (over the set S):
$$z_i = d_i + (e_i \cdot \rho_i) + \lambda_i \cdot s_i \cdot c;$$
4. securely deletes $((d_i, D_i), (e_i, E_i))$, then returns z_i to SA.

Finally SA recomputes ρ_i, $R_i = D_i \cdot (E_i)^{\rho_i}$, $R = \prod_{i \in S} R_i$ and c, and verifies every share against the public data:
$$g^{z_i} \stackrel{?}{=} R_i \cdot Y_i^{c \cdot \lambda_i},$$
identifying and reporting the misbehaving participant and aborting if any check fails; otherwise it publishes $\sigma = (R, z)$ with $z = \sum_{i \in S} z_i$.

Correctness. The proof is two interpolations (paper §6.1). The key polynomial $F_1(x) = \sum_{j=1}^{n} f_j(x)$ from key generation has $s_i = F_1(i)$ and $s = F_1(0)$. The nonce values $(i, (d_i + e_i \cdot \rho_i)/\lambda_i)$ define a degree-$(\alpha - 1)$ polynomial $F_2(x)$ with $F_2(0) = \sum_{i \in S} d_i + e_i \cdot \rho_i = k$. Setting $F_3(x) = F_2(x) + c \cdot F_1(x)$, each response is $z_i = \lambda_i F_3(i)$, so $z = \sum_{i \in S} z_i$ interpolates $F_3(0) = k + c \cdot s$; with $R = g^k$ and $c = H_2(R, Y, m)$, the pair (R, z) is a correct Schnorr signature on m.

2.4 Why the binding factor exists: Drijvers and ROS

The one non-obvious ingredient is $\rho_i = H_1(i, m, B)$. The binding factor is FROST's answer to a pair of concurrency attacks that broke or bounded every prior two-round scheme.

The Drijvers attack. Against two-round Schnorr multisignatures in which the adversary may open T parallel sessions, Drijvers et al. showed how to forge by finding a hash output that is a sum of others: $c^* = H(R^*, Y, m^*)$ with $c^* = \sum_{j=1}^{T} H(R_j, Y, m_j)$, using Wagner's k-tree algorithm for the generalised birthday problem in time $O(\kappa \cdot b \cdot 2^{b/(1+\lg \kappa)})$ for $\kappa = T + 1$ and b the bitlength of the group order. The attack transfers from the n-of-n multisignature setting to a t-of-n threshold scheme with an adversary controlling up to $t-1$ signers, and it is why the Gennaro et al. scheme must cap its signing parallelism. Benhamouda et al.'s subsequent polynomial-time ROS solver (for $\ell > \lg q$ parallel sessions) turned the same structural weakness from subexponential to practical against schemes including two-round MuSig and GJKR-DKG-based threshold Schnorr; their paper concludes that "the current version of" FROST is *not* affected.

How binding forecloses it. Because each signer derives $\rho_i = H_1(i, m, B)$ before responding, its share z_i is bound to this message, this signing set, and this exact commitment list. Responses cannot be recombined across sessions, messages, or reordered signer sets: any such mix changes some ρ_ℓ, hence R, hence c, and the transplanted share fails verification. The paper's own generalisation (its Equation 1, with linear combinations γ_j) states the residual attack surface precisely: the forger must find
$$H_2(R^*, Y, m^*) = \sum_j \gamma_j \cdot H_2(R_j, Y, m_j),$$
where $R^* = g^{r^*} \cdot \prod_j \widehat{R}_j^{\gamma_j}$ is itself a function of every input on the right-hand side. Wagner's algorithm needs the two sides of the collision to vary independently; here every candidate combination on the right moves the left-hand target, degrading the generalised birthday collision into multiple preimage attacks. The paper's footnote 7 concedes this step is heuristic: sufficiency of the condition is immediate, necessity is not proved.

2.5 Nonce hygiene

The preprocessed pairs (d_{ij}, e_{ij}) are one-time keys in the strictest sense. The paper's stipulations:

- each pair is used at most once, and the signer *securely deletes* it in the same signing step that returns z_i (Figure 3, step 6);
- an accidentally reused (d_{ij}, e_{ij}) can expose the long-term share s_i;
- a SA that maliciously replays a commitment pair achieves nothing: the participant simply aborts, so replay grants no power;
- the commitment store (if a server) is trusted only for availability and freshness; possession of the public (D_{ij}, E_{ij}) lists grants no signing power.

See §2.8 for RFC 9591's sharpening of nonce generation (hedged sampling, and an explicit warning that deterministic nonces enable complete key recovery in the multi-party setting).

2.6 The security theorem as stated

The paper proves EUF-CMA security, in the random oracle model, by reduction to the *discrete logarithm problem* in $\mathbb{G}$ — deliberately not to one-more discrete log (the adversary must return the discrete logarithms of $\ell + 1$ challenges after at most ℓ queries to a discrete-log oracle), so that the negative result of Drijvers et al., which rules out security reductions from that assumption for a family of two-round Schnorr multisignatures, does not apply. The proof is for an intermediate scheme, *FROST-Interactive*, in which the binding value is produced by a one-time verifiable random function $\rho_i = a_{ij} + b_{ij} \cdot H_\rho(m, B)$ with committed keys.

> **Theorem 6.1** (ePrint 2020/852). *If the discrete logarithm problem in $\mathbb{G}$ is (τ', ϵ')-hard, then the FROST-Interactive signature scheme over $\mathbb{G}$ with n signing participants, a threshold of t, and a preprocess batch size of π is $(\tau, n_h, n_p, n_s, \epsilon)$-secure whenever*
>
> $$\epsilon' \leq \frac{\epsilon^2}{2n_h + (\pi + 1)n_p + 1} \quad \text{and}$$
>
> $$\tau' = 4\tau + \big(30\pi n_p + (4t - 2)n_s + (n + t - 1)t + 6\big) \cdot t_{\text{exp}} + O(\pi n_p + n_s + n_h + 1),$$
>
> *such that t_{exp} is the time of an exponentiation in $\mathbb{G}$, assuming the number of participants compromised by the adversary is less than the threshold t.*

Here n_h, n_p, n_s bound the forger's random-oracle, preprocess, and signing queries. The architecture is Bellare–Neven generalised forking: the DL challenge ω is embedded as the honest participant's free-coefficient commitment ϕ_{t0} in a simulated KeyGen, the simulator is run twice on the same tape with oracle answers diverging from index J, and two forgeries yield $\text{dlog}(Y) = (z' - z)/(h'_J - h_J)$. The VRF is what lets the simulator win the programming race: it can compute R before the forger does and program $H_2(R, Y, m)$ with success probability 1.

The step from FROST-Interactive to plain FROST (replace the VRF by $\rho_i = H_1(i, m, B)$) is argued *heuristically* in the paper's §6.2, not proven — the Equation-1 gap of §2.4. This gap is what the later literature closed, differently and under a different assumption (§2.7).

2.7 The corrected and extended analyses

Three later results matter for how this volume cites FROST's security.

Crites–Komlo–Maller (ePrint 2021/1375). This work, "How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures" (IACR ePrint 2021/1375, received 2021-10-12, last revised 2022-08-03), introduced the optimisation now called *FROST2*: a single binding factor $d = h_1(vk, lr)$ shared by all signers, replacing the original per-signer $d_i = h_1(vk, lr, i)$ (that original is retroactively *FROST1*) and reducing the signers' scalar multiplications from linear in the number of signers to constant.

Bellare–Crites–Komlo–Maller–Tessaro–Zhu (CRYPTO 2022). The paper "Better than Advertised Security for Non-interactive Threshold Signatures" — its FROST/BLS part maintained as Bellare–Tessaro–Zhu, ePrint 2022/833, June 2022 — gave the analysis now cited by RFC 9591. The authors define a hierarchy TS-UF-0 $\leq \cdots \leq$ TS-UF-4 (and strong variants TS-SUF-2/3/4) grading what counts as a trivial forgery. Their results, proven in the ROM under the *one-more discrete logarithm* (OMDL) assumption, without the AGM, and in the trusted-dealer setting (the DKG is explicitly not modelled):

- FROST1 is TS-SUF-3-secure, and is *not* TS-UF-4-secure;
- FROST2 is TS-SUF-2-secure, and is *not* TS-UF-3-secure — the shared binding factor loses the guarantee that the signer set that committed is the signer set that signed;
- concretely (their Theorem 5.1, verbatim bound), for a TS-SUF-2 adversary A making at most q_s preprocessing and q_h random-oracle queries there is an OMDL adversary B making at most $2q_s + ns$ challenge queries (ns their number of parties) with

$$\mathrm{Adv}^{\text{ts-suf-2}}_{\mathrm{FROST2}[\mathbb{G}]}(A) \leq \sqrt{q \cdot \left(\mathrm{Adv}^{\text{omdl}}_{\mathbb{G}}(B) + 3q^2/p\right)}, \qquad q = q_s + q_h + 1,$$

  and an analogous Theorem 5.4 for FROST1 at TS-SUF-3.

The security FROST actually deploys with is therefore: OMDL + ROM, static corruptions below t, trusted-dealer keys — a different basis than the original paper's DL-based proof of the interactive variant plus heuristic bridge.

Gouvêa–Komlo (ePrint 2024/436). The preprint "Re-Randomized FROST" (received 2024-03-13) is the design ZIP 312 builds on: FROST unchanged, plus algorithms GenRand ($\alpha \xleftarrow{\$} \mathbb{Z}_p$), RandShare ($\bar{x}_i \leftarrow x_i + \hat{\alpha}$ with $\hat{\alpha} = H_r(\alpha, \mathrm{aux})$, aux an optional external-protocol transcript contribution), RandPKShare ($\overline{PK}_i = PK_i \cdot g^{\hat{\alpha}}$) and RandPK ($\overline{PK} = PK \cdot g^{\hat{\alpha}}$). Correctness rests on the $\sum_{i \in S} \lambda_i = 1$ identity, giving $z = r + c(x + \hat{\alpha})$. Its Theorem 1 proves unforgeability in the ROM under *algebraic* OMDL (AOMDL) — the same assumptions as plain FROST per BCKMTZ — with the bound (its Equation 3)

$$\mathrm{Adv}^{\text{sec}}_{TS,A} \leq \sqrt{q \cdot \mathrm{Adv}^{\text{aomdl}}_{D}(\lambda) + 2q^2/p - \mathrm{negl}(\lambda)}, \qquad q = q_h + q_s,$$

via the local forking lemma; the randomizer being public lets the reduction strip it ($z_0 \leftarrow z^* - c^* H_r(\alpha^*, \mathrm{aux}^*)$) during extraction. The coordinator chooses the randomizer and is trusted for privacy and unlinkability only, not for unforgeability. The full treatment of this paper belongs to the re-randomisation chapter (§3).

2.8 RFC 9591 against the paper: what the standard changed

RFC 9591 (IRTF CFRG, Informational, June 2024; Connolly, Komlo, Goldberg, Wood) is FROST as implementations ship it, including the `frost-core` 3.0.0 workspace this volume cites. Reconciling it against ePrint 2020/852:

- *Two rounds only.* The RFC specifies the two-round protocol; the paper's single-round-with-preprocessing variant (batch parameter π, commitment server) is declared out of scope.

- *No DKG.* The paper's Figure-1 KeyGen is dropped; key generation is out of scope, with *trusted-dealer* generation given in Appendix C only. The BCKMTZ analysis the RFC cites matches this framing (it, too, models a trusted dealer).
- *Binding factor inputs widened.* The paper binds $\rho_i = H_1(i, m, B)$. The RFC computes, per participant,

  $$\texttt{binding\_factor}_i = H_1(\texttt{PK\_enc} \,\|\, H_4(\texttt{msg}) \,\|\, H_5(\texttt{encoded commitment list}) \,\|\, \texttt{enc}(i)),$$

  adding the *group public key* to the hash and pre-hashing the message and commitment list. The per-participant structure means RFC 9591 standardises FROST1: the RFC's §7.2 rules the single-binding-factor (FROST2) optimisation NOT RECOMMENDED, citing the loss of the started-set-equals-signed-set guarantee (the TS-SUF-3 vs TS-SUF-2 separation of §2.7).
- *Hedged nonces.* The paper samples (d, e) uniformly; the RFC hedges: each nonce is $H_3(r \,\|\, \texttt{enc}(sk_i))$ with r a fresh 32-byte random string, so a weak RNG is backstopped by the key share. It warns that fully deterministic derivation enables complete key recovery in the multi-party setting.
- *Five domain-separated oracles.* H_1 (binding factor), H_2 (challenge), H_3 (nonce), H_4 (message pre-hash), H_5 (commitment-list pre-hash), each with a per-ciphersuite `contextString`; the challenge input order `group_comm_enc` $\|$ `PK_enc` $\|$ `msg` equals the paper's $H_2(R, Y, m)$.
- *Roles renamed and channels stated.* The signature aggregator becomes the *Coordinator*; the channel is assumed authenticated. The Coordinator SHOULD verify the aggregate signature and MAY verify individual shares — the paper's per-share check $g^{z_i} = R_i \cdot Y_i^{c\lambda_i}$ survives as verify_signature_share — with identifiable abort, as in the paper, and no robustness.
- *Ciphersuites.* The RFC ships FROST(Ed25519, SHA-512), FROST(ristretto255, SHA-512) (RECOMMENDED), FROST(Ed448, SHAKE256), FROST(P-256, SHA-256), and FROST(secp256k1, SHA-256); no Zcash suite appears in the standard itself. The two Zcash suites, FROST(Jubjub, BLAKE2b-512) and FROST(Pallas, BLAKE2b-512), are instead defined by ZIP 312, whose H_2 personalisation `Zcash_RedPallasH` makes the FROST challenge equal RedDSA's $\mathsf{H}^{\circledast}$ — the precise sense in which the standardised protocol plugs into the spend-authorisation verifier of the *Ironwood Guide*, §"Spend authorisation: RedPallas re-randomisation".
- *Security claim.* The RFC claims EUF-CMA per BCKMTZ, citing both the original paper and BCKMTZ — i.e., the deployed claim rests on the OMDL/ROM analysis, not the paper's DL-based theorem for the interactive variant.

Classification: FROST as in RFC 9591 and its Zcash ciphersuites as in ZIP 312 are *specified*; ZIP 312 itself remains Draft status (zips commit 6961098410, 2026-07-09), with its key-generation procedure explicitly out of scope and PedPoP DKG for Zcash deployment *designed-but-unspecified* (implemented in `frost-core`'s `dkg` module but not standardised in any ZIP or RFC).

3 Re-Randomized FROST for RedPallas

The FROST of §2, standardised as RFC 9591 (§2.8), signs for a *fixed* group public key: the aggregate (R, z) verifies under the single-signer Schnorr equation for PK, and under nothing else. Zcash spend authorisation verifies under no fixed key at all. Every Sapling Spend and every Orchard Action carries a freshly randomised validating key, and consensus checks the spend-authorisation signature against that per-Action key — a key which, as §3.1 makes precise, no set of FROST shares interpolates to unless every share is shifted. This chapter presents the design that closes the gap: the Re-Randomized

FROST of Gouvêa and Komlo (IACR ePrint 2024/436), its Zcash specification in ZIP 312, the ciphersuite FROST(Pallas, BLAKE2b-512), the deployed implementation in the `frost-rerandomized` and `reddsa` crates, and the custody surfaces the construction serves.

A rigor note before the material. RFC 9591 is the only artefact here at standards rigor. The Gouvêa–Komlo paper is a preprint — received 2024-03-13, no peer-reviewed venue — and ZIP 312 carries Draft status (zips commit 6961098410, 2026-07-09). Following the series rule, every claim beyond the RFC is binned as *specified*, *designed but unspecified*, or an *open problem*; the closing classification is in §3.7.

3.1 The mismatch: consensus verifies only randomised keys

The protocol specification (§4.15, spend authorization signatures) prescribes, for every Sapling Spend and every Orchard Action, the following signer-side flow. The signer chooses a fresh spend-auth randomizer α and:

1. samples $\alpha \leftarrow \mathsf{SpendAuthSig.GenRandom}()$;
2. derives $\mathsf{rsk} = \mathsf{SpendAuthSig.RandomizePrivate}(\alpha, \mathsf{ask})$;
3. derives $\mathsf{rk} = \mathsf{SpendAuthSig.DerivePublic}(\mathsf{rsk})$;
4. proves the Spend or Action statement with α in the auxiliary (secret) input and rk in the primary input;
5. signs: $\mathsf{spendAuthSig} = \mathsf{SpendAuthSig.Sign}_{\mathsf{rsk}}(\mathsf{SigHash})$.

Consensus therefore verifies the signature under $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, never under ak itself.

The randomisation algorithms are RedDSA's, from the protocol specification (§5.4.7):

$$\begin{aligned} \mathsf{RedDSA.RandomizePrivate}(\alpha, \mathsf{sk}) &:= \mathsf{sk} + \alpha \pmod{r_{\mathbb{G}}}, \\ \mathsf{RedDSA.RandomizePublic}(\alpha, \mathsf{vk}) &:= \mathsf{vk} + [\alpha]\, G, \end{aligned}$$

with identity randomizer 0; RedDSA.GenRandom chooses T uniformly among byte sequences of length $(\ell_H + 128)/8$ — 80 bytes at $\ell_H = 512$ — and returns $H^{\circledast}(T)$, where $H^{\circledast}(B) := \mathrm{LEOS2IP}_{\ell_H}(H(B)) \bmod r_{\mathbb{G}}$. Signing draws its nonce as $r = H^{\circledast}(T \,\|\, \mathsf{vk}^{\star} \,\|\, M)$ for fresh uniform T, sets $R = [r]\, G$ and $S = \big(r + H^{\circledast}(R^{\star} \,\|\, \mathsf{vk}^{\star} \,\|\, M) \cdot \mathsf{sk}\big) \bmod r_{\mathbb{G}}$, and outputs $\sigma = R^{\star} \,\|\, S^{\star}$. Validation recomputes $c = H^{\circledast}(R^{\star} \,\|\, \mathsf{vk}^{\star} \,\|\, M)$ and accepts iff $R \neq \bot$, $S < r_{\mathbb{G}}$, $R^{\star}$ is canonical (post-ZIP-216), and $[h_{\mathbb{G}}]\big(-[S]\, G + R + [c]\, \mathsf{vk}\big) = 0_{\mathbb{G}}$. RedPallas specialises the template with $\mathbb{G}$ the Pallas group, $\ell_H = 512$, and $H(x) = \text{BLAKE2b-512}(\texttt{Zcash\_RedPallasH}, x)$; $\mathsf{SpendAuthSig}^{\mathsf{Orchard}}$ uses the generator $G^{\mathsf{SpendAuth}} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard}, \texttt{"G"})$ and "is instantiated as RedPallas with key re-randomization". The security notion the protocol requires of the scheme is SURK-CMA (Strong Unforgeability with Re-randomized Keys), adapted from Fleischhacker et al. (PKC 2016, §3), whose oracle answers queries of the form (m, α) — message and adversary-chosen randomizer together.

The construction and its single-party security are settled in the *Ironwood Guide*: §"Spend authorisation: RedPallas re-randomisation" builds $\mathsf{rsk} := \mathsf{ask} + \alpha$ and $\mathsf{rk} := \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, and §"RedPallas unforgeability" proves EUF-CMA under DL in the ROM *including* adversary-chosen α, by key-prefixed forking with $\mathsf{ask} = \mathsf{rsk}^{*} - \alpha^{*}$. Per the layering rule we cite those results and do not re-prove them.

The threshold problem is now visible. A FROST sharing of ask gives shares sk_i with $\sum_{i \in S} \lambda_i \mathsf{sk}_i = \mathsf{ask}$ for every signing set S; every signature that coalition can produce verifies under ak. But no honest transaction ever asks for a signature under ak: the verifier's key is rk, shifted by a fresh α per Action. The shares must be shifted too — and, as the next subsection shows, the correct shift is the *same* α for every participant, not a Shamir sharing of α.

3.2 Re-Randomized FROST: the construction

The design is due to Gouvêa and Komlo (both Zcash Foundation), "Re-Randomized FROST", IACR ePrint 2024/436, received 2024-03-13; we work from the full text, with the construction figure and both displayed equations verified against page renders of the PDF. As in §2 we quote the paper in its own multiplicative notation (g^x for $[x]\,G$, $PK \cdot g^{\hat{\alpha}}$ for $PK + [\hat{\alpha}]\,G$); the additive RedPallas form returns with ZIP 312 in §3.4.

Definition 3.1 (Perfectly rerandomizable threshold scheme; ePrint 2024/436, Definition 5)**.** A threshold signature scheme

$$TS = (\mathsf{Setup}, \mathsf{KeyGen}, (\mathsf{Sign}, \mathsf{Sign}'), \mathsf{Combine}, \mathsf{Verify})$$

is *perfectly re-randomizable* if there exist additional PPT algorithms GenRand, RandShare, RandPKShare, and RandPK, and a randomness distribution X, such that the Rand* algorithms take a randomizer $\alpha \in X$ and an auxiliary string $\mathrm{aux} \in \{0,1\}^*$.

The paper's Remark 1 explains aux: it models an optional contribution from an external protocol transcript, so that a new session changes the re-randomised key even under a repeated α.

Construction 3.2 (Re-Randomized FROST; ePrint 2024/436, Figure 5)**.** The additions to plain FROST — the paper's grey-boxed lines — are exactly four algorithms:

$$\begin{aligned}
\mathsf{GenRand}() &: \alpha \xleftarrow{\$} \mathbb{Z}_p;\\
\mathsf{RandShare}(x_i, \alpha, \mathrm{aux}) &: \hat{\alpha} \leftarrow H_r(\alpha, \mathrm{aux}); \quad \bar{x}_i \leftarrow x_i + \hat{\alpha};\\
\mathsf{RandPKShare}(PK_i, \alpha, \mathrm{aux}) &: \overline{PK}_i \leftarrow PK_i \cdot g^{\hat{\alpha}};\\
\mathsf{RandPK}(PK, \alpha, \mathrm{aux}) &: \overline{PK} \leftarrow PK \cdot g^{\hat{\alpha}}.
\end{aligned}$$

The FROST core runs unchanged on the shifted inputs. Algorithm Sign() samples $(r_k, s_k) \xleftarrow{\$} \mathbb{Z}_p^2$ and commits $(R_k, S_k) = (g^{r_k}, g^{s_k})$. Algorithm Sign$'$, given the randomised share $\bar{x}_k$ and randomised key $\overline{PK}$ in place of x_k and PK, derives each binding factor a_i by H_{non} from the participant identifier, the message, $\overline{PK}$, and the full commitment list $\{(i, R_i, S_i)\}_{i \in S}$, then computes

$$R \leftarrow \prod_{i \in S} R_i \cdot S_i^{a_i}, \qquad c \leftarrow H_{\mathrm{sig}}(\overline{PK}, R, m), \qquad z_k \leftarrow r_k + s_k a_k + c\,\lambda_k \bar{x}_k.$$

Algorithm Combine outputs $z \leftarrow \sum_{i \in S} z_i$, $\sigma \leftarrow (R, z)$; Verify$(\overline{PK}, m, \sigma)$ recomputes the challenge and accepts iff $R \cdot \overline{PK}^{\,c} = g^z$.[a]

[a] The paper's own argument order for H_{sig} is inconsistent: Sign$'$ and Combine write $H_{\mathrm{sig}}(PK, R, m)$, while Verify and the reduction's table write $H_{\mathrm{sig}}(PK, m, R)$ — and both differ from the order RFC 9591 (§4.6) and RedDSA fix, $c = H_2(R^\star \,\|\, PK^\star \,\|\, m)$, which ZIP 312 and the deployed `reddsa` crate use. We read the paper's orderings as a presentational slip with no normative weight; the deployed order is RedDSA's. The paper also twice refers to "Figure 4" where Figure 5 is meant.

The paper is explicit that this is the whole design: "We do not make any changes to the plain FROST protocol." The only interface change is which share and which public key Sign$'$ is handed.

Correctness is one identity. The paper's Equation (2):

$$z = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c \sum_{i \in S} \lambda_i \bar{x}_i = \sum_{i \in S} r_i + \sum_{i \in S} s_i a_i + c\Big(\sum_{i \in S} \lambda_i x_i + \hat{\alpha} \sum_{i \in S} \lambda_i\Big) = r + c\,(x + \hat{\alpha}) = r + c\,\hat{x}, \quad (1)$$

"because x is Shamir secret shared" and "$\sum_{i \in S} \lambda_i = 1$ since it is the interpolation of the constant

polynomial $f(X) = 1$". The output is a regular re-randomised signature for $\overline{PK} = PK \cdot g^{\hat{\alpha}}$ with $\hat{\alpha} = H_r(\alpha, \text{aux})$.

The identity deserves emphasis, because it is the entire reason the construction is this simple. Every participant adds the *same* $\hat{\alpha}$ to its share — no dealing, no per-participant shares of the randomizer — and the Lagrange weights, summing to one over any interpolating set, carry the common shift through interpolation so that the group secret moves by $\hat{\alpha}$ rather than by $|S| \cdot \hat{\alpha}$. Had the weights summed to anything else, or varied with S in a way the signers could not predict, the shifted shares would interpolate to a key nobody computed.

3.3 Unforgeability: TRUF, AOMDL, and Theorem 1

The game. The paper's unforgeability notion (Definition 6, game TRUF, its Figure 4) extends threshold unforgeability with adversarial randomizers. Corruption is *static*: the adversary fixes its up-to-$(t-1)$ corrupt parties before key generation (the paper titles its Figures 2 and 4 "static unforgeability games"). The signing oracle $\mathcal{O}^{\mathsf{Sign}'}(k, \text{ssid}, m, S, \alpha, \text{aux}, \{\rho_i\})$ lets the *adversary* choose the randomizer and the auxiliary string per query; a forgery $(m^*, \sigma^*, \alpha^*, \text{aux}^*)$ wins if m^* was never queried and σ^* verifies under PK *or* under $\mathsf{RandPK}(PK, \alpha^*, \text{aux}^*)$. The unrandomised notion is the special case randomizer $= 0$. The lineage is Fleischhacker et al. (PKC 2016) — the same single-party re-randomisable-key framework, and re-randomised-Schnorr EUF result, that the protocol specification's SURK-CMA definition adapts (§3.1).

The assumption. AOMDL is the *algebraic one-more discrete logarithm* game (the paper's Figure 1, taken from the MuSig2 paper): the adversary receives $\ell + 1$ challenges, may query a discrete-log oracle only on linear combinations $(\alpha, \{\beta_i\})$ of environment-generated elements, and must return all $\ell + 1$ discrete logarithms having made at most ℓ queries. The algebraic restriction on oracle queries is what §2.7's OMDL does not impose.

The theorem. Verbatim (Theorem 1 with Equation (3); the radical verified against the rendered page):

> **Theorem 1** (ePrint 2024/436). *Rerandomized-FROST is unforgeable in the random oracle model, assuming the AOMDL assumption holds in* $\mathbb{G}$, concretely
>
> $$\text{Adv}^{\text{sec}}_{TS,\mathcal{A}} \le \sqrt{q \cdot \text{Adv}^{\text{aomdl}}_{D}(\lambda) + 2q^2/p - \text{negl}(\lambda)}\,, \qquad q = q_h + q_s, \tag{2}$$
>
> with q_h random-oracle and q_s signing queries.

Remark 3.3 (Placement of the negligible term)**.** The printed bound places $-\,\text{negl}(\lambda)$ *inside* the square root; we transcribe it as printed. The customary form of such forking-lemma bounds — including the BCKMTZ bound quoted in §2.7 — carries additive slack outside the radical, and a negative term inside is dimensionally odd for an advantage bound.

Proof shape. The reduction $\mathcal{B}$ receives $2q_s + 1$ AOMDL challenges and sets $PK = X_0$. It simulates trusted-dealer key generation by sampling the corrupt parties' shares and producing every verification share by Lagrange interpolation in the exponent, so that PK_i is implicitly $g^{f(i)}$ without $\mathcal{B}$ knowing any honest share. Round-one answers are challenge pairs $R_i = X_i, S_i = X_{i+1}$. In round two the reduction runs RandPK and $\mathsf{RandPKShare}$ honestly — it cannot run $\mathsf{RandShare}$, holding no share — and derives each honest signature share in the exponent, as $Z_k = R_i \cdot S_i^{a_k} \cdot \widehat{PK}_k^{\,c}$, querying the AOMDL linear-combination oracle for the scalar z_k; the simulation is perfect. The local forking lemma of Bellare–Dai–Li (ASIACRYPT 2019) reprograms H_{sig} at the forgery's challenge query. The randomizers, being public, are stripped before extraction: when $\alpha \neq \perp$,

$$z_0 = z^* - c^* H_r(\alpha^*, \text{aux}^*), \qquad z_1 = z^{**} - c^{**} H_r(\alpha^{**}, \text{aux}^{**}), \qquad y_0 = \frac{z_0 - z_1}{c^* - c^{**}},$$

and the remaining challenge logarithms are extracted as in the FROST proof of Bellare–Crites–Komlo–Maller–Tessaro–Zhu (CRYPTO 2022; §2.7). The count closes the one-more game: $2q_s$ oracle queries against $2q_s + 1$ extracted logarithms.

What the theorem does not say. Four caveats frame every citation of this result. First, the paper proves *unforgeability only*: it contains no formal unlinkability definition and no unlinkability theorem; that property is argued at the ZIP layer (§3.4) by matching RedDSA's distributions, under the protocol specification's SURK-CMA framing. Second, corruptions are static. Third, the threshold-scheme definition assumes centralised (trusted-dealer) key generation, which the paper says "can [be] adapted" to a DKG — adapted, not proven. Fourth, the paper's §5.1 sketches how a DKG-style contribution round could remove the trusted-for-privacy coordinator (all parties contribute to the randomizer, none learns it) at the cost of extra communication rounds; ZIP 312 does not adopt this. The cost of re-randomisation itself is one fixed-base scalar multiplication per participant and per coordinator; the paper's Table 1 benchmarks (Pallas suite, Ryzen 9 5900X) put the signing overhead at 83% with 2 signers, 45% at 7, and 8% at 67.

3.4 ZIP 312: the specification

ZIP 312, "FROST for Spend Authorization Multisignatures" (Status Draft, Category Wallet, created 2022-08; owners Gouvêa, Komlo, Connolly; zcash/zips PR #662), specifies Re-Randomized FROST as a delta against RFC 9591. Its target property beyond unforgeability is *unlinkability*, which the ZIP defines as statistical independence of signatures from the signers' long-term keys: for uniform randomizers and absent metadata leakage, it is "impossible to determine whether two transactions were generated by the same party".

The modifications to RFC 9591. Five, and only five:

- *Randomizer generation.* A new helper `randomizer_generate(msg, commitment_list)`: draw `rng_randomizer` ← `random_bytes(Ns)`, form

  ```
  randomizer_input = rng_randomizer ||
                     encode_signing_package(commitment_list, msg),
  ```

  and return H_R(`randomizer_input`). Implementations MAY use another encoding "as long as all values (the message, and the identifier, binding nonce and hiding nonce for each participant) are unambiguously encoded".

- *Round one* is unchanged.

- *Round two.* The Coordinator generates the randomizer and sends it to each signer "over a confidential and authenticated channel" — plain FROST requires authentication only — together with the message and the commitment list. Each participant then runs `sign` with sk_i + randomizer in place of its share and PK + [randomizer] G in place of the group public key.

- *Aggregation* shifts the group key the same way: PK += [randomizer] G.

- *Share verification* shifts both keys: PK_i += [randomizer] G and PK += [randomizer] G in `verify_signature_share`.

The ZIP's "randomizer" is the paper's *effective* randomizer $\hat{\alpha}$: per the paper's §7 implementation notes, GenRand and H_r are folded into the single hash call above, and the value distributed to signers is the already-hashed scalar.

The algebra, in RedDSA notation. The ZIP's Rationale restates the correctness identity of (1) additively, and it is worth displaying because it is the entire consensus-facing content of the design. A valid RedDSA response must satisfy $z = r + c \cdot \mathsf{sk}$. Each FROST share contributes

$$z_i = \big(\texttt{hiding\_nonce}_i + \texttt{binding\_nonce}_i \cdot \rho_i\big) + \lambda_i \cdot c \cdot \mathsf{sk}_i,$$

and under re-randomisation the key terms become $c \cdot (\mathsf{sk} + \text{randomizer})$ on the single-signer side and, summed over the coalition,

$$\sum_{i \in S} \lambda_i \, c \, (\mathsf{sk}_i + \text{randomizer}) = c\Big(\sum_{i \in S} \lambda_i \mathsf{sk}_i + \text{randomizer} \cdot \sum_{i \in S} \lambda_i\Big) = c\,(\mathsf{sk} + \text{randomizer}),$$

since $\sum_i \lambda_i \mathsf{sk}_i = \mathsf{sk}$ by Shamir and $\sum_i \lambda_i = 1$ (the ZIP proves the latter in its `sum-lambda-proof` footnote, citing math.SE 1325342). Every participant adds the same randomizer to its share; the weights-sum-to-one identity is exactly why the group secret shifts by α and not by $|S| \cdot \alpha$.

Who learns the randomizer. The threat model is the ZIP's sharpest departure from RFC 9591. The Coordinator generates the randomizer and is "trusted with the privacy of the transaction (which includes the unlinkability property)" but *not* with unforgeability: "a rogue Coordinator ... should not be able to create signed transactions without the approval of `MIN_PARTICIPANTS` participants". All key-share holders receive the randomizer and are likewise trusted with privacy only. The role assignment is natural: the Coordinator fits the transaction builder, who already creates the zero-knowledge proof — and the proof needs α as auxiliary input (step 4 of the flow in §3.1). The confidentiality requirement is not optional hygiene: anyone learning $\hat{\alpha}$ can un-randomise the on-chain key, $\mathsf{ak} = \mathsf{rk} - [\hat{\alpha}]\,G$, so the paper advises encrypting the randomizer "so that eavesdroppers can't break unlinkability". Non-requirements are stated with equal care: the coordinator-less variant of RFC 9591 §7.5 is unsupported;[2] share holders can always link the signing operation to the eventual on-chain transaction; key generation is out of scope; network privacy is out of scope.

The message is a hash. The signed message is the SigHash, which conveys nothing about the transaction to a signer reading it. Signers MUST therefore check the SigHash against transaction data sent over the same channel, or recompute it themselves; the mechanism is out of the ZIP's scope. This is the threshold echo of a rule the single-signer flow gets for free: a signer who cannot see what it signs authorises whatever the builder chose.

Unlinkability, and where it is proven. Nowhere, formally: the paper proves unforgeability only (§3.3), and the ZIP argues unlinkability in its Rationale by matching distributions — "`randomizer_generate` generates `randomizer` uniformly at random as required by `RedDSA.GenRandom`", and the signing flow is compatible with RedDSA.RandomizePrivate, RedDSA.RandomizePublic, RedDSA.Sign, and RedDSA.Validate — while its Requirements defer the formal notion to the protocol specification's SURK-CMA definition. The distribution-matching argument does connect to a proved statement: the *Ironwood Guide*'s §"RedPallas unforgeability" shows honest uniform re-randomisations are distributed as fresh Schnorr keys. What no source supplies is a threshold-specific unlinkability theorem. Bin: the mechanism is *specified* (ZIP 312, Draft); its unlinkability guarantee rests on a distribution argument, not a theorem.

3.5 The ciphersuite FROST(Pallas, BLAKE2b-512)

ZIP 312 defines the ciphersuite in RFC 9591's template. The group is Pallas with base point $G^{\mathsf{SpendAuth}}$ as in protocol §5.4.7.1, of order p_{Vesta}; `RandomScalar` is uniform in $[0, \texttt{Order}-1]$; `SerializeElement`

[2] ZIP 312's reference for the variant mislabels it as RFC 9591 "Section 7.3: Removing the Coordinator Role"; the section is 7.5 (§7.3 is "Nonce Reuse Attacks"), and the reference's URL anchor targets the correct section.

is the point compression $\text{repr}_{\mathsf{Pallas}}$ ($P^\star$ in series notation) and `DeserializeElement` is $\text{abst}_{\mathsf{Pallas}}$, erroring on $\perp$ *and on the identity element*; `SerializeScalar` is little-endian over 32 bytes and `DeserializeScalar` rejects values $\geq$ the order. The hash is BLAKE2b-512 with 16-byte personalisation strings and $N_h = 64$; scalar-valued hashes interpret the 64 output bytes as a little-endian integer reduced mod the order. Table 2 lists the oracles.

Oracle	Role	Personalisation
H_1	binding factor	`FROST_RedPallasR`
H_2	challenge	`Zcash_RedPallasH`
H_3	nonce	`FROST_RedPallasN`
H_4	message pre-hash	`FROST_RedPallasM`
H_5	commitment-list pre-hash	`FROST_RedPallasC`
H_R	randomizer	`FROST_RedPallasA`

Table 2: FROST(Pallas, BLAKE2b-512) hash personalisations (ZIP 312). The challenge hash reuses RedPallas's own personalisation; the FROST additions use the `FROST_` prefix.

The single load-bearing row is H_2: ZIP 312 states it is "equivalent to $H^{\circledast}(m)$" of RedPallas, so the FROST challenge *is* the on-chain challenge, and signature verification for the suite is RedPallas validation itself. The ZIP's compatibility rationale completes the argument that a FROST aggregate is a plain RedPallas signature: the (R, S) output split matches RedDSA.Validate's parsing; FROST and Zcash use the same challenge input order (R, public key, message); and the fact that r and R are FROST-generated rather than produced by RedDSA.Sign's nonce derivation is irrelevant to a verifier, which sees only the distribution of R. A parallel suite FROST(Jubjub, BLAKE2b-512) — personalisations `FROST_RedJubjub{R,N,M,C,A}` and `Zcash_RedJubjubH` — serves Sapling spend authorisation; this volume works the Pallas suite and treats the Jubjub twin as its mechanical transliteration.

3.6 Deployed implementation: `frost-rerandomized` and `reddsa`

Claims in this subsection cite ZcashFoundation/frost at commit `2016e44` (2026-04-23, workspace version 3.0.0) and ZcashFoundation/reddsa at commit `3792daa` (2026-05-22, crate version 0.5.2), examined 2026-07-15. The pinned commits are the published releases — tag `frost-core/v3.0.0` (crates.io 2026-04-23) and `reddsa` 0.5.2 — so main-at-commit and the released crates coincide. ZIP 312 names `reddsa` as the reference implementation of both ciphersuites. A current-head check on 2026-08-30 found frost `0966bd1` and the same reddsa head `3792daa`; the seven later FROST commits update the Rust edition, minimum compiler, dependencies, and CI, not the re-randomized protocol derived below.

The generic layer. Crate `frost-rerandomized` (`frost-rerandomized/src/lib.rs`) implements Construction 3.2 over any `frost-core` ciphersuite. Trait impl `Randomize<KeyPackage>` computes the randomised signing share as `signing_share + randomizer` and the randomised verifying share by adding the `randomizer_element`; struct `RandomizedParams` carries the `randomizer` ("also called alpha" in its documentation), `randomizer_element` $=$ `[randomizer]`G, and `randomized_verifying_key` $=$ PK $+$ `randomizer_element`. The current flow differs instructively from a literal reading of the ZIP: the Coordinator sends a randomizer *seed*, not the randomizer, and each participant recomputes the randomizer from the seed and its own round-one commitments. The derivation, its divergence from the draft text — the message is no longer a hash input — and the reconciliation in flight in zips PR #895 are treated once, in §4.2.

The RedPallas suite. Module `src/frost/redpallas.rs` in `reddsa` implements the `frost-core` `Ciphersuite` trait for `PallasBlake2b512` (contextual ID `"FROST(Pallas, BLAKE2b-512)"`). Its `generator()` is `orchard::SpendAuth::basepoint()`, whose constant bytes are reproducible

as `pallas::Point::hash_to_curve("z.cash:Orchard")(b"G")` — the protocol's $G^{\mathsf{SpendAuth}}$. Oracles $H_1/H_3/H_4/H_5$ carry the `FROST_RedPallas{R,N,M,C}` personalisations of Table 2; H_2 goes through the crate's `HStar` default, `Zcash_RedPallasH`; the `RandomizedCiphersuite::hash_randomizer` impl uses `FROST_RedPallasA` (the ZIP's H_R); scalar reduction is the 64-byte-wide `from_bytes_wide` (`src/hash.rs`), whose Pallas implementation delegates to pasta's `from_uniform_bytes` (`src/orchard.rs`). Beyond ZIP 312, the crate defines two further personalisations — `HDKG = FROST_RedPallasD` and `HID = FROST_RedPallasI`, the DKG and identifier hashes — for key generation, which the ZIP does not specify at all.

Even-Y normalisation. One deployment detail appears in neither the paper nor the ZIP 312 body: the `reddsa` hooks `post_generate`/`post_dkg` normalise fresh key material to the even-Y representative the protocol specification requires of ak; the full mechanism is §4.1. The *randomised* key rk need not satisfy the even-Y condition — only ak does.

The RedPallas re-randomised API. Module `src/frost/redpallas/rerandomized.rs` re-exports the re-randomised flow for the suite — `sign_with_randomizer_seed()`, `aggregate()`, and `aggregate_custom()` — together with the type aliases `Randomizer` and `RandomizedParams` over `PallasBlake2b512`. The module README's worked example runs the full trusted-dealer plus re-randomised signing flow and ends by verifying the aggregate under the `randomized_verifying_key()` of its `RandomizedParams` — the executable form of this chapter's claim that a FROST aggregate is a plain RedPallas signature under rk.

3.7 Custody surfaces and classification

(a) Spend-authorisation custody. The primary surface: a t-of-n sharing of ask signing Orchard Actions (and, via the Jubjub twin suite, Sapling Spends). FROST replaces step 5 of the signer flow in §3.1; steps 1–4 move to the Coordinator-as-builder, who draws the randomizer, proves the Action statement with α as auxiliary input, and publishes rk — the circuit and consensus rules are untouched, per the *Ironwood Guide*, §"Spend authorisation: RedPallas re-randomisation". Bin: *specified* (ZIP 312, Draft), with key generation explicitly out of the ZIP's scope; DKG-based key generation for Zcash remains *designed but unspecified* (implemented in `frost-core`, standardised nowhere), as §2.8 already concluded for the base protocol. Related anticipations elsewhere in the ZIP corpus: ZIP 316 requires FROST unified viewing key material to follow ZIPs 312 and 2005, and ZIP 271 anticipates lockbox disbursement to "a FROST address", noting (as of May 2025) that ZIP 312 does not specify key generation.

(b) ZIP 2005 quantum spending key custody. ZIP 2005's "Usage with FROST" section constrains threshold deployments of its quantum-recovery hierarchy: the key ak is derived jointly, by FROST DKG, from participants' shares of ask, and the participants privately agree on a value sk from which nk, qsk, qk, and $\mathsf{rivk}_{\mathsf{ext}}$ derive. The full ceremony, its derivations, and its asymmetric threat model — every participant holds qsk, so the threshold property does not survive a quantum adversary — are developed once, in §4.7. Bin: the FROST-facing requirements are *specified* (ZIP 2005's requirements list calls for full compatibility with ZIP 312 FROST, hardware wallets, and their combination); the combined DKG-plus-shared-sk key generation is *designed but unspecified* — no source specifies the protocol by which participants "privately agree" on sk.

(c) Crosslink finalizer keys. The *Crosslink Guide*, §"Finalizer keys", bins threshold custody of finalizer keys as an *open problem*, and we adopt that bin. One precision for this volume: the applicable protocol there is *plain* FROST — the finalizer design has no key re-randomisation, so the natural suite is RFC 9591's FROST(Ed25519, SHA-512), not either ZIP 312 suite. The prototype state and this observation are developed once, in §4.8.

(d) PCZT signing. The multiparty transaction flow that would host the Coordinator role is the PCZT of the *Wallet Guide*, §"Partially created transactions (PCZT)": ZIP 374 (Partially Created Zcash Transaction Format, Draft, owner Jack Grigg) names threshold signing ("for example FROST", per RFC 9591) in its Motivation among the multiparty use cases its Signer role must accommodate.

Classification. In the series' three bins: base FROST and its five ciphersuites are *specified* at standards rigor (RFC 9591, §2.8); Re-Randomized FROST, its threat model, and both Zcash ciphersuites are *specified* by ZIP 312 at Draft status, with the unforgeability theorem resting on a preprint (ePrint 2024/436, no venue) and unlinkability on a distribution argument rather than a theorem; key generation — FROST DKG for Zcash, and ZIP 2005's shared-sk agreement — is *designed but unspecified*; threshold custody of Crosslink finalizer keys is an *open problem*. Deployed-behaviour claims (seed-based randomizer derivation, even-Y normalisation, the extra DKG personalisations) are implementation facts of the pinned commits, cited as such.

4 Deployment and open questions

The preceding sections dealt in designs and proofs; this one deals in what ships. We pin the exact code path a Zcash threshold-custody deployment exercises today, reconcile it against the draft ZIP text where the two have diverged, then walk the three custody surfaces this volume targets — spend authorisation through PCZT, the ZIP-2005 quantum spending key, and Crosslink finalizer keys — and close by sorting every artefact into the series' three bins (*specified*, *designed-but-unspecified*, *open problem*), each with the concrete trigger on which its classification must be revisited.

4.1 The deployed stack

Two Zcash Foundation repositories carry the implementation. The `frost` workspace (commit `2016e44ba4`, 2026-04-23, tagged `frost-core/v3.0.0`; workspace version 3.0.0, MSRV 1.81) provides the generic `frost-core`, the RFC 9591 ciphersuite crates `frost-ristretto255`, `-ed25519`, `-ed448`, `-p256`, `-secp256k1`, and `-secp256k1-tr`, and the generic re-randomisation layer `frost-rerandomized`. Beyond RFC 9591's two-round protocol the workspace implements trusted-dealer key generation (the RFC's appendix), the Pedersen-with-knowledge-proof DKG "as specified in the original paper" (ePrint 2020/852, §2.2 — not in the RFC), Repairable-Threshold-Scheme share recovery (ePrint 2017/1155), share refresh, and Re-Randomized FROST (ePrint 2024/436, §3). The README declares the crates "considered stable and feature complete", with SemVer guarantees.

The `frost-rerandomized` crate is `no_std`; its `Randomize` implementations add the randomizer group element to every verifying share of a `PublicKeyPackage` and the randomizer scalar to the signing share of a `KeyPackage`, whose `randomize` documentation warns: "You MUST NOT reuse the randomized key package for more than one signing." Its README notes that "the main ciphersuite crates do not re-expose the rerandomization functions... The exceptions are the Zcash ciphersuites in `reddsa` which do expose the randomized functionality" — the re-randomised layer exists, in shipped form, for Zcash alone.

The `reddsa` crate (commit `3792daa95e`, 2026-05-22, version 0.5.2) supplies those ciphersuites: its feature `frost = ["frost-rerandomized", "alloc"]` pulls `frost-rerandomized` 3.0.0, and its modules `reddsa::frost::redpallas` and `reddsa::frost::redjubjub` implement the two ZIP-312 ciphersuites (`PallasBlake2b512`, `JubjubBlake2b512`), each with a `keys` module (trusted-dealer, DKG, repairable) and a `rerandomized` submodule re-exporting `sign_with_randomizer_seed`, `aggregate`, `aggregate_custom`, `Randomizer`, and `RandomizedParams`. Function `hash_randomizer` instantiates ZIP 312's H_R as $H^{\circledast}$ (the code's `HStar`) under the personalisation `FROST_RedPallasA` (respectively `FROST_RedJubjubA`), matching §3.5.

The RedPallas module additionally enforces the even-y requirement that ZIP 2005's amended protocol-spec §4.2.3 places on `ak` (§4.7): trait `keys::EvenY` negates the group verifying key, every

verifying share, the signing shares, and the VSS commitment coefficients whenever the serialised key has $\tilde{y} = 1$ (the code's evenness test is `serialized[31] & 0x80 == 0`), and the `post_generate` and `post_dkg` ciphersuite hooks call `into_even_y` on fresh key material, so trusted-dealer and DKG outputs alike land on the even-y representative. Negation commutes with Shamir sharing because it is linear. The RedJubjub module carries no such machinery; the constraint is Pallas/Orchard-specific.

4.2 The randomizer flow: draft ZIP against shipped crate

ZIP 312 ("FROST for Spend Authorization Multisignatures"; owners Gouvea, Komlo, Connolly; Status Draft; created 2022-08) specifies, at zips commit `6961098410`, the randomizer generation

$$\texttt{signing\_package\_enc} = \texttt{encode\_signing\_package}(\texttt{commitment\_list}, \texttt{msg}),$$
$$\texttt{randomizer} = H_R\big(\texttt{random\_bytes}(N_s) \,\|\, \texttt{signing\_package\_enc}\big),$$

with H_R for FROST(Pallas, BLAKE2b-512) being BLAKE2b-512 under the personalisation `FROST_RedPallasA`, output reduced to a Pallas scalar (an element of $\mathbb{F}_{p_{\mathsf{Vesta}}}$); H_2 is `Zcash_RedPallasH`, i.e. exactly RedPallas's $H^{\circledast}$ (§3.5). For Jubjub the personalisations are `FROST_RedJubjub{R,N,M,C,A}` and `Zcash_RedJubjubH`, with base point G^{Sapling}. Round 2 then proceeds as plain FROST with three substitutions,

$$sk_i \leftarrow sk_i + \texttt{randomizer}, \qquad PK_i \leftarrow PK_i + \texttt{ScalarBaseMult}(\texttt{randomizer}),$$
$$\textit{group\_public\_key} \leftarrow \textit{group\_public\_key} + \texttt{ScalarBaseMult}(\texttt{randomizer}),$$

the second applying in share verification. The Coordinator sends the randomizer over a "confidential and authenticated channel" — strictly stronger than plain FROST's authenticated-only channel assumption, because the randomizer links the ceremony to the on-chain rk.

The deployed crate has moved past this text. Crate `frost-rerandomized` 3.0.0 deprecates `Randomizer::new(rng, signing_package)` — the exact draft-ZIP flow above, message included — in favour of `Randomizer::new_from_commitments`: the Coordinator draws an N_s-byte `randomizer_seed`, sends the *seed*, and each participant regenerates

$$\texttt{randomizer} = \texttt{hash\_randomizer}\big(\texttt{seed} \,\|\, \texttt{encode\_group\_commitments}(\texttt{commitments})\big)$$

locally via `regenerate_from_seed_and_commitments`. The message is no longer an input, and — the crate's stated rationale — participants no longer need to fully trust the Coordinator's RNG, since their own round-one commitments enter the hash. In the paper's terms (ePrint 2024/436, Definition 5 and Remark 1), the commitment encoding plays the role of the optional contributory string aux, which models "the transcript of some external protocol"; the paper's own §7 implementation notes describe the older design, with the message still included, in the same terms.

The gap between draft and crate is being closed in zips PR #895 ("[ZIP 312] Specify key generation, change randomizer handling"; opened August 2024, 21 commits, still open as of 2026-07-15, awaiting review by daira and nuttycom). It adds a key-generation specification, revises the randomizer handling (the randomizer "couldn't possibly be generated using the message as an input"), integrates the ZIP-2005 qsk material, renames the scheme consistently to "Re-Randomized FROST", and adds Daira-Emma Hopwood as a ZIP owner. Its commit history also retires the draft's serialisation reference — the ZF FROST book's moving-target serialisation-format page — in favour of RFC 9591's serialisation function. Until it merges, this volume presents the crate flow — message omission included — as the observed fact of the pinned commit, draft lag rather than a conformance defect, and the ZIP text as in flight.

The trust geometry is worth stating exactly. The paper's §5.1 holds that the central randomizer-choosing party is "trusted with preserving privacy of the scheme" but "not trusted with security", and that removing it would require a DKG-style contributory sub-protocol at extra rounds and complexity. ZIP 312's threat model matches: the Coordinator is trusted with transaction privacy and unlinkability

but cannot forge without `MIN_PARTICIPANTS` approvals, and every key-share holder is trusted with transaction privacy. Non-requirements are equally explicit: ZIP 312 does not remove the Coordinator role, does not (at this commit) provide key generation, and places network privacy out of scope.

4.3 PCZT as the integration surface

ZIP 374's Motivation names "multiparty transactions — threshold signing (for example FROST [RFC 9591])" among the problems the PCZT format solves; the *Wallet Guide*, §"Partially created transactions (PCZT)", documents the format and its role graph. The insertion point for FROST is the Signer role (*Wallet Guide*, §"Signing"): to sign an Orchard spend the action's re-randomisation scalar α must be present, the local-key path computes $\mathsf{rsk} := \mathsf{ask}+\alpha$ — the re-randomisation of the *Ironwood Guide*, §"Spend authorisation: RedPallas re-randomisation" — and checks $\mathsf{VerificationKey}(\mathsf{rsk}) = \mathsf{rk}$, and the *external* path, `apply_orchard_signature`, accepts a signature if and only if it verifies under the action's rk over the shielded sighash (`InvalidExternalSignature` otherwise). A FROST-aggregated RedPallas signature is exactly such an external signature; for it to verify, the Coordinator must use the PCZT action's α as the randomizer, so that the re-randomised group key RandPK produces equals that action's rk.

PCZT v2 has since shipped in `pczt` 0.8.0. Current `pczt` 0.9.3 carries separate Orchard and Ironwood bundles and tags an externally produced `SpendAuthSignature` with its value pool and action index; `apply_orchard_spend_auth_signature` dispatches to the corresponding bundle and verifies under that action's rk. The invariant in the preceding paragraph — one shared shielded sighash and the action's own α — is unchanged for v6 Ironwood spends (librustzcash `91f448b5`, `pczt/src/roles/signer/mod.rs`).

The end-to-end flow has been demonstrated (ZF FROST book, `zcash/devtool-demo.md`):

1. `zcash-devtool pczt create` produces the PCZT;
2. the `zcash-sign` tool (from ZcashFoundation/frost-zcash-demo, now `frost-tools`) prints a SIGHASH and a `Randomizer` from the transaction plan;
3. `frost-client` runs the FROST protocol over `frostd` with that SIGHASH as the message and that randomizer;
4. `zcash-sign sign` writes the aggregated signature into `frost_pczt.signed`;
5. `zcash-devtool pczt combine` merges the signed and proven PCZTs;
6. `zcash-devtool pczt send` broadcasts.

Here `zcash-devtool` builds on `zcash_keys` with the features `"orchard"`, `"unstable"`, and `"unstable-frost"`. The `unstable-frost` feature (`= ["orchard"]`) exposes the constructor `UnifiedFullViewingKey::from_orchard_fvk` — a UFVK from an Orchard FVK alone, since a FROST wallet has no unified spending key — and the `zcash_keys` changelog (0.3.0, 2024-08-19) frames the flag as existing "to temporarily expose API features that are needed specifically when creating FROST threshold signatures", to be removed "once key derivation for FROST has been fully specified and implemented". Repository `librustzcash` itself, that is, classifies FROST key derivation as not fully specified.

One check remains outside every specification: ZIP 312 requires that signers "MUST check that the given SIGHASH matches the data sent from the Coordinator, or compute the SIGHASH themselves", and explicitly leaves the mechanism out of scope. In the demonstrated flow the SIGHASH travels from `zcash-sign` to the participants through `frost-client`; by what mechanism a shareholding participant gains independent confidence in it is unspecified.

4.4 Wallet-integration constraints

The ZF FROST book's technical-details chapter records the constraints a FROST-capable Zcash wallet inherits. FROST cannot cover transparent inputs, which authorise by ECDSA; the book accordingly suggests supporting Orchard only ("Orchard is simpler to handle"). With a DKG-generated ak, the remaining Orchard key components derive from ak plus either an sk used only for nk and rivk, or independently generated nk and rivk. A seed phrase alone can never restore a FROST wallet: restoration needs the key share, all verifying shares, the participant identifiers, and the FVK components, so share backup is a JSON-file affair and share recovery falls to the Repairable Threshold Scheme (§4.1).

The Foundation's own status report ("The State of FROST for Zcash", 2025-05-20) names wallet integration the missing piece ("The missing piece is for wallets to integrate FROST"): no mainstream wallet ships FROST, and "technical-inclined users can use FROST with Zcash today using our `frost-client` command line tool". The same post lists the "lack of a standardized key generation specification for FROST" as the interoperability limiter, flags metadata protection (e.g. Tor) as needing work, and offers ZF as a candidate operator of a production `frostd`.

4.5 Transport: frostd and frost-client

The reference transport is `frostd`, a JSON-HTTP session server: clients authenticate with key pairs, the Coordinator creates sessions naming the participants' public keys, and all FROST messages are end-to-end encrypted, so the server itself is untrusted. The CLI `frost-client` drives it; both live in ZcashFoundation/frost-tools (renamed from `frost-zcash-demo`). Least Authority, as Zcash Ecosystem Security Lead, audited `frost-client` and `frostd` (announcement 2025-05-01) with no high-severity findings, all findings addressed on `main`. No ZIP specifies this transport; ZIP 312 places network privacy out of scope (§4.2).

4.6 Audit coverage, precisely

Two audits exist, and neither covers the code path Zcash custody would run. NCC Group audited the v0.6.0 release (commit `5fa17ed`) of `frost-core`, `frost-ed25519`, `frost-ed448`, `frost-p256`, `frost-secp256k1`, and `frost-ristretto255` — including trusted-dealer and DKG key generation and FROST signing — with all findings addressed and reviewed by NCC (public report 2023-10-23). The repository README is explicit: "This does not include `frost-secp256k1-tr` and rerandomized FROST." Earlier, the 2021 "Audit of FROST implementation" (JP Aumasson and Adrien Hamelink, Taurus; Omer Shlomovits, ZenGo; dated 2021-03-23) covered the pre-rewrite implementation in the RedJubjub repository's `src/frost.rs` ($\approx$ 404 lines at commit `2ebc08f`, 2021-02-25) — two-round FROST with a trusted dealer only, RedJubjub only — concluding "We did not find any major security issue."

The composition, therefore: `frost-rerandomized` plus the `reddsa` RedPallas/RedJubjub ciphersuites — the exact stack of §4.1 — has no completed third-party audit, and the only audit ever to touch a Zcash FROST ciphersuite predates the `frost-core` rewrite.

4.7 Custody of the ZIP-2005 quantum spending key

ZIP 2005 ("Ironwood Quantum Recoverability"; owners Daira-Emma Hopwood and Jack Grigg; Status Proposed, Category Consensus; activation at the NU6.3 activation height) is the second custody surface. Its "Usage with FROST" section derives ak jointly, from the participants' shares of ask, via FROST DKG, and then constrains the ceremony: participants "MUST privately agree on a value sk, and then use it with use_qsk = true to derive nk, qsk, qk, and (using the ak output by the DKG protocol) rivk_ext"; the protocol MUST ensure all participants obtain the same ak, nk, and rivk_ext. The

derivations, verbatim from the ZIP:

$$\mathsf{H}^{\mathsf{qsk}}(\mathsf{sk}) = \text{truncate}_{32}\big(\mathsf{PRFexpand}_{\mathsf{sk}}([\texttt{0x0C}])\big),$$
$$\mathsf{H}^{\mathsf{qk}}(\mathsf{qsk}) = \text{BLAKE3.derive\_key}(\texttt{"Zcash ZIP 2005 qk-derivation v1"}, \mathsf{qsk}, 32),$$
$$\mathsf{H}^{\mathsf{rivk\_ext}}_{\mathsf{qk}}(\mathsf{ak}, \mathsf{nk}) = \text{ToScalar}^{\mathsf{Orchard}}\big(\mathsf{PRFexpand}_{\mathsf{qk}}([\texttt{0x0D}] \,||\, \text{I2LEOSP}_{256}(\mathsf{ak}) \,||\, \text{I2LEOSP}_{256}(\mathsf{nk}))\big),$$

with the PRFexpand domain-separator bytes `0x0A–0x0D` registered by ZIP 2005. In the use_qsk = true branch of the amended protocol-spec §4.2.3, the wallet obtains $\mathsf{ak}^{\mathbb{P}}$ "by any suitably secure method that ensures the last bit of $\text{repr}_{\mathbb{P}}(\mathsf{ak}^{\mathbb{P}})$ is 0" — the requirement the `reddsa EvenY` machinery of §4.1 enforces — and sets $\mathsf{ak} = \text{Extract}_{\mathbb{P}}(\mathsf{ak}^{\mathbb{P}})$, $\mathsf{qsk} = \mathsf{H}^{\mathsf{qsk}}(\mathsf{sk})$, $\mathsf{qk} = \mathsf{H}^{\mathsf{qk}}(\mathsf{qsk})$, and $\mathsf{rivk} = \mathsf{H}^{\mathsf{rivk\_ext}}_{\mathsf{qk}}(\mathsf{ak}, \mathsf{nk})$. The ZIP's list of admissible generation methods for $\mathsf{ak}^{\mathbb{P}}$ explicitly includes "FROST distributed key generation [zip-0312-key-generation] in which the corresponding spend-authorization material is *t*-of-*n* shared among the participants".

The threat model shifts accordingly. The host wallet in a multi-signer FROST setup is the ZIP-312 Coordinator; it holds nk, ak, and qk. With FROST and hardware wallets, spend authorisation breaks only by (1) possession of qsk *plus* finding discrete logarithms on Pallas — and "the adversary is an authorized FROST signer — they hold a copy of qsk by construction" is the ZIP's variant 1.a — or (2) breaking RedDSA, FROST, the DKG, or the proving system. Each participant MUST treat qsk as a secret held at least as securely and reliably as ask: it is not needed until the Recovery Protocol, but losing it can lose funds once the current Orchard protocol is disabled. A quantum adversary holding only qsk — which every participant holds — may steal funds, so the ZIP RECOMMENDS that once a fully post-quantum threshold multisignature protocol exists, all FROST-held funds migrate to its pool. FROST's key advantage — signing with shares without ever reconstructing ask — is retained through the Recovery Protocol, which continues to require a RedDSA signature.

One tension is unresolved. FROST DKG exists precisely to avoid a commonly-known secret, yet the ceremony above requires the participants to "privately agree on a value sk" from which nk and qsk derive — and neither ZIP 2005 nor the crates give a protocol for that agreement. ZIP 2005's Deployment section points at PR #895 ("There is no significant existing deployment of FROST, so we can write this into ZIP 312 from the start"), so the resolution is expected there: as of 2026-07-15 the PR's commit history drafts a contributory sk-generation section (2026-03-05), still unmerged, so the shared-sk agreement stays *designed but unspecified* with #895's merge as its trigger.

4.8 Crosslink finalizer keys

The third surface is prospective only. The *Crosslink Guide*, §"Finalizer keys", records that the prototype uses single ed25519 keys per finalizer (ed25519-zebra), with 76-byte votes signed by appending a 64-byte ed25519 signature over those 76 bytes, and that no FROST or threshold-signing design for finalizer keys is sourced anywhere in the Crosslink corpus (negative search over tfl-book, crosslink-protocol-guide, crosslink-spec, and zebra-crosslink, dated 2026-07-15 in that guide); key rotation, revocation, threshold custody, and initial-roster formation are classified open problems in every design source.

We record one observation of our own, clearly marked as such: finalizer votes are plain ed25519, with no key re-randomisation, so the fitting ciphersuite would be RFC 9591's FROST(Ed25519, SHA-512) via the NCC-audited `frost-ed25519` crate, with no re-randomisation layer — finalizer keys are deliberately long-lived roster identities, not unlinkable one-time keys, so the entire apparatus of §3 is unnecessary there. No published design says this; it is this volume's inference from the vote format and the audit table.

4.9 Classification and revisit triggers

Per the series' three-way scheme:

- *Specified.* FROST as in RFC 9591 (§2.8); Re-Randomized FROST for spend-authorisation custody, per ZIP 312 at Draft status — PR #895's in-flight randomizer and key-generation changes are its named revisit trigger and do not move the bin (§4.2); the `frost-core`/`frost-rerandomized`/`reddsa` API surface at the pinned versions (§4.1); the PCZT Signer external-signature path (§4.3).
- *Designed-but-unspecified.* FROST key generation for Zcash (out of scope at the pinned ZIP-312 commit; the crates implement the PedPoP DKG of ePrint 2020/852, but no ZIP specifies it); the ZIP-2005 use_qsk FROST key-generation constraints (ZIP Proposed, referencing the unmerged #895; the sk-agreement step has no protocol, §4.7); the transport (`frostd` exists and is audited, but no ZIP covers it, §4.5).
- *Open problem.* Crosslink finalizer threshold custody (§4.8); post-quantum threshold signing for the qsk-era migration (§4.7); hardware-wallet FROST share custody — ZIP 2005's threat model assumes hardware-wallet FROST signers, but no device implements a share signer.

Two former revisit triggers have fired without changing these bins. NU6.3 is active, but `zcash_keys` still gates `from_orchard_fvk` behind `unstable-frost` pending a specified FROST derivation. PCZT v2 is released, and the current signer preserves the α and external-signature surface for Ironwood as described in §4.3. The remaining classifications expire on the following identifiable events, in rough order of expected impact:

- *PR #895 merges.* The randomizer flow, the key-generation specification, the sk-agreement protocol, and the ZIP-312/ZIP-2005 integration all re-derive from the merged text; the serialisation reference it pins replaces the ZF-FROST-book moving target (§4.2).
- *An audit of the re-randomised stack is announced.* The "no completed third-party audit" claim of §4.6 must be rechecked immediately before any printing in any case.
- *A finalizer key-management design is published.* The Crosslink bin moves, and the `frost-ed25519` observation is either confirmed or discarded (§4.8).
- *A post-quantum threshold multisignature protocol exists.* ZIP 2005's RECOMMENDED migration of FROST-held funds becomes actionable (§4.7).